

1.PUZZLE PROBLEM

Code:

```
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]

print("Enter 9 numbers:")

state = list(map(int, input().split()))

misplaced = 0

for i in range(len(goal)):

    if state[i] != goal[i] and state[i] != 0:

        misplaced += 1

print("\nInitial State:")

for i in range(0, 9, 3):

    print(state[i], state[i+1], state[i+2])

print("\nGoal State:")

for i in range(0, 9, 3):

    print(goal[i], goal[i+1], goal[i+2])

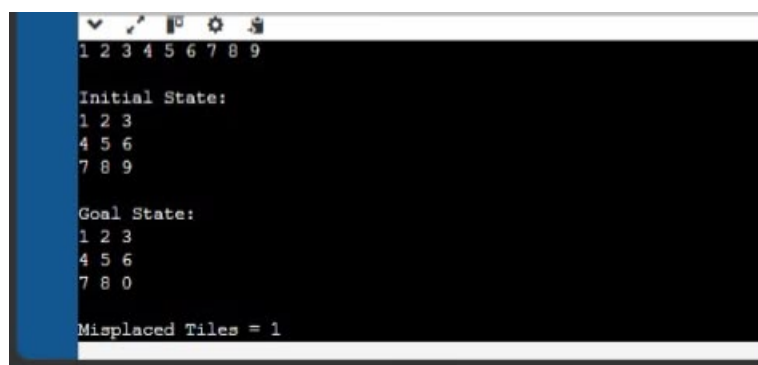
if state == goal:

    print("\nGoal State Reached!")

else:

    print("\nMisplaced Tiles =", misplaced)
```

OUTPUT



```
1 2 3 4 5 6 7 8 9

Initial State:
1 2 3
4 5 6
7 8 9

Goal State:
1 2 3
4 5 6
7 8 0

Misplaced Tiles = 1
```

2.QUEEN PROBLEM

Code:

```
n = int(input("Enter number of queens: "))

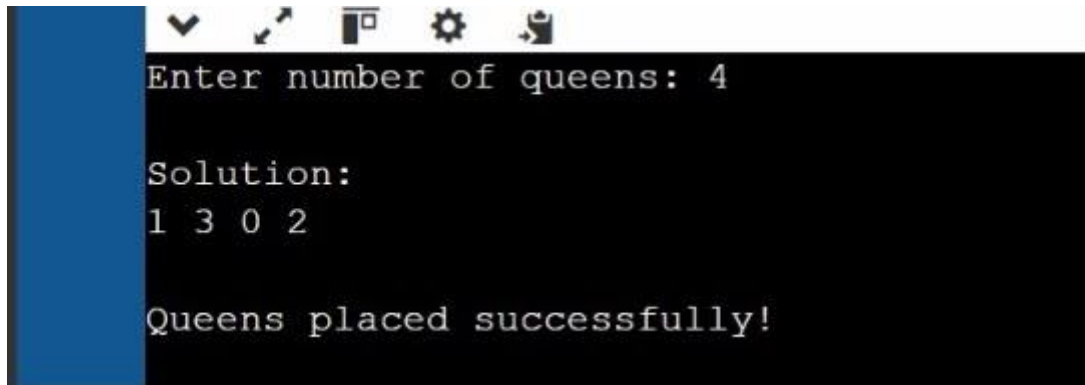
board = [-1] * n

def is_safe(row, col):
    for j in range(col):
        if board[j] == row:
            return False
        if abs(board[j] - row) == abs(j - col):
            return False
    return True

def solve(col):
    if col == n:
        print("\nSolution:")
        print(*board)
        return True
    for row in range(n):
        if is_safe(row, col):
            board[col] = row
            if solve(col + 1):
                return True
    board[col] = -1
    return False

if solve(0):
    print("\nQueens placed successfully!")
else:
    print("\nNo Solution")
```

OUTPUT:



```
Enter number of queens: 4

Solution:
1 3 0 2

Queens placed successfully!
```

3.WATER JUG PROBLEM

Code:

```
jug1 = int(input("Enter Jug 1 capacity: "))
jug2 = int(input("Enter Jug 2 capacity: "))
target = int(input("Enter target amount: "))

x = 0
y = 0

while y != target:
    if x == 0:
        x = jug1
    elif y == jug2:
        y = 0
    else:
        amount = min(x, jug2 - y)
        x = x - amount
        y = y + amount

print("Jug 1 =", x)
print("Jug 2 =", y)
print("Target Reached!")
```

OUTPUT:

A terminal window with a dark background and a blue vertical bar on the left. The window title bar is visible at the top with standard icons. The text inside the terminal is as follows:

```
Enter Jug 1 capacity: 4
Enter Jug 2 capacity: 3
Enter target amount: 2
Jug 1 = 0
Jug 2 = 2
Target Reached!
```

4.VACUUM CLEANER PROBLEM

Code:

```
room_a = input("Enter status of Room A: ")
room_b = input("Enter status of Room B: ")
print("\nInitial State:")
print("A =", room_a)
print("B =", room_b)
print("\nVacuum is in Room A")
if room_a.lower() == "dirty":
    print("Cleaning Room A...")
    room_a = "Clean"
else:
    print("Room A is clean")
print("\nVacuum moves to Room B")
if room_b.lower() == "dirty":
    print("Cleaning Room B...")
    room_b = "Clean"
```

else:

```
    print("Room B is clean")
```

```
print("\nFinal State:")
```

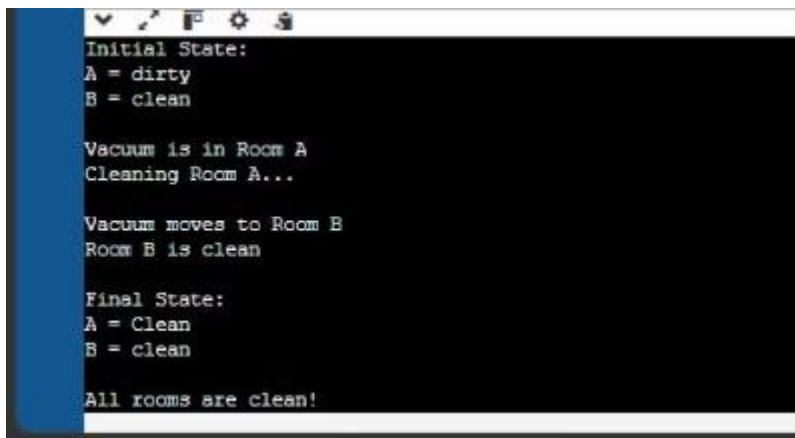
```
print("A =", room_a)
```

```
print("B =", room_b)
```

```
if room_a.lower() == "clean" and room_b.lower() == "clean":
```

```
    print("\nAll rooms are clean!")
```

Output:



```
Initial State:  
A = dirty  
B = clean  
  
Vacuum is in Room A  
Cleaning Room A...  
  
Vacuum moves to Room B  
Room B is clean  
  
Final State:  
A = Clean  
B = clean  
  
All rooms are clean!
```

5.BREADTH FIRST SEARCH (BFS)

Code:

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D'],  
    'C': ['A', 'D'],  
    'D': ['B', 'C']  
}  
  
def bfs(start):  
    visited = []  
    queue = [start]  
    while queue:  
        node = queue.pop(0)  
        if node not in visited:  
            print(node, end=" ")  
            visited.append(node)  
            for next_node in graph[node]:  
                if next_node not in visited:  
                    queue.append(next_node)  
  
print("BFS Traversal:")  
  
bfs('A')
```

OUTPUT:



The screenshot shows a terminal window with a dark background and a blue sidebar on the left. The terminal output is as follows:

```
BFS Traversal:  
A B C D  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

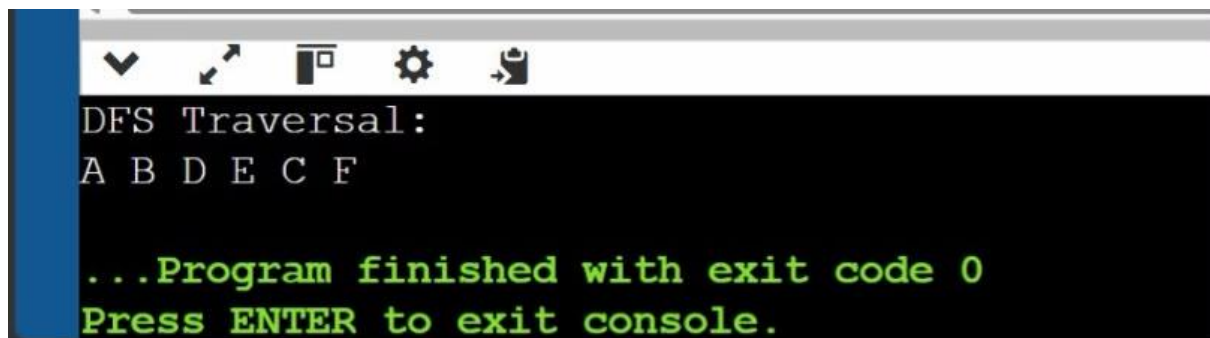
The terminal window has a title bar with standard icons (minimize, maximize, close, settings, and a user icon) and the text "inp" in the top right corner.

6.DEPTH FIRST SEARCH (DFS)

Code:

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}  
  
visited = set()  
  
def dfs(node):  
    if node in visited:  
        return  
    print(node, end=" ")  
    visited.add(node)  
    for next_node in graph[node]:  
        dfs(next_node)  
  
print("DFS Traversal:")  
dfs('A')
```

Output:

A screenshot of a terminal window with a dark background and a blue vertical bar on the left. The terminal shows the output of a DFS traversal. The first line is "DFS Traversal:" followed by the nodes "A B D E C F" on the next line. Below this, a green message states "...Program finished with exit code 0" and "Press ENTER to exit console." The terminal window has a standard toolbar at the top with icons for window management and settings.

```
DFS Traversal:  
A B D E C F  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

7.MISSIONARIES AND CANNIBALS PROBLEM

Code:

```
from collections import deque

start = (3, 3, 0)

goal = (0, 0, 1)

queue = deque()

queue.append((start, []))

visited = set()

moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

while queue:

    state, path = queue.popleft()

    if state in visited:

        continue

    visited.add(state)

    if state == goal:

        print("Solution:")

        for s in path + [state]:

            print(s)

        break

    m, c, boat = state

    for dm, dc in moves:

        if boat == 0:

            new_state = (m - dm, c - dc, 1)

        else:

            new_state = (m + dm, c + dc, 0)

    nm, nc, nb = new_state

    if 0 <= nm <= 3 and 0 <= nc <= 3:
```



```

        if (nm == 0 or nm >= nc) and \
            (3-nm == 0 or 3-nm >= 3-nc):
queue.append(
    (new_state, path + [state])
)

```

Output:

```

Solution:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)

```

8. TRAVELLING SALESMAN PROBLEM

Code:

```

distance = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

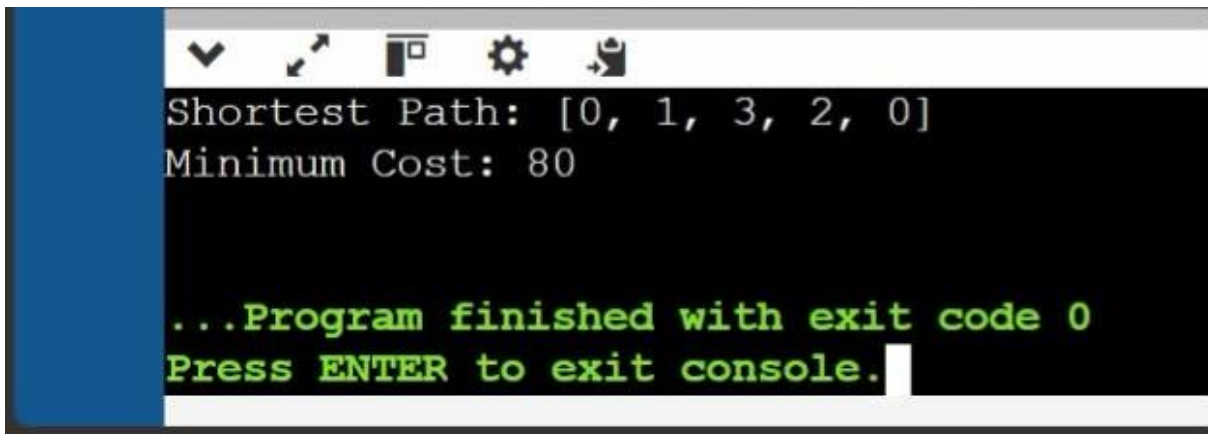
min_cost = 9999
best_route = []

for a in range(1, 4):
    for b in range(1, 4):

```

```
        for c in range(1, 4):
if a != b and b != c and a != c:
total = (distance[0][a] +
        distance[a][b] +
        distance[b][c] +
        distance[c][0])
if total < min_cost:
    min_cost = total
    best_route = [0, a, b, c, 0]
print("Shortest Path:", best_route)
print("Minimum Cost:", min_cost)
```

OUTPUT:



```
Shortest Path: [0, 1, 3, 2, 0]
Minimum Cost: 80

...Program finished with exit code 0
Press ENTER to exit console.
```

9.CRYPT ARITHMETIC PROBLEM

Code:

```
from itertools import permutations

digits = range(10)

for p in permutations(digits, 8):

    S = p[0]
    E = p[1]
    N = p[2]
    D = p[3]
    M = p[4]
    O = p[5]
    R = p[6]
    Y = p[7]

    if S != 0 and M != 0:

        SEND = S*1000 + E*100 + N*10 + D

        MORE = M*1000 + O*100 + R*10 + E

        MONEY = M*10000 + O*1000 + N*100 + E*10 + Y

        if SEND + MORE == MONEY:

            print("SEND =", SEND)

            print("MORE =", MORE)

            print("MONEY =", MONEY)

            break
```

OUTPUT:



```
MORE = 1085
MONEY = 10652

...Program finished with exit code 0
Press ENTER to exit console.
```

10.A* ALGORITHM

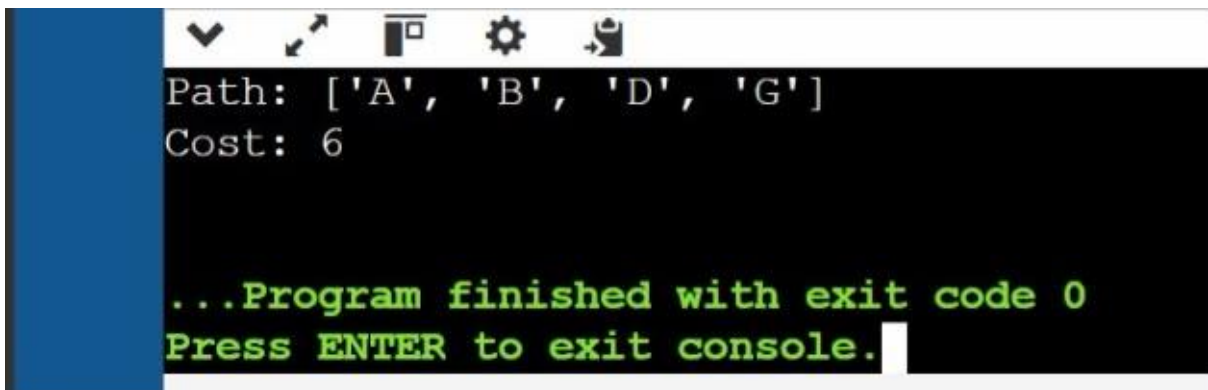
Code:

```
graph = {  
    'A': {'B': 1, 'C': 4},  
    'B': {'D': 2, 'E': 5},  
    'C': {'E': 1},  
    'D': {'G': 3},  
    'E': {'G': 2},  
    'G': {}  
}  
  
h = {  
    'A': 6,  
    'B': 4,  
    'C': 3,  
    'D': 2,  
    'E': 1,  
    'G': 0  
}  
  
start = 'A'  
goal = 'G'  
open = [start]  
cost = {start: 0}  
parent = {start: None}  
  
while open:  
    x = min(open, key=lambda n: cost[n] + h[n])  
    open.remove(x)  
  
if x == goal:  
    path = []
```

```
while x:
    path.append(x)
    x = parent[x]
print("Path:", path[::-1])
print("Cost:", cost[goal])
break

for y in graph[x]:
    new_cost = cost[x] + graph[x][y]
if y not in cost or new_cost < cost[y]:
    cost[y] = new_cost
    parent[y] = x
    if y not in open:
        open.append(y)
```

OUTPUT:



```
Path: ['A', 'B', 'D', 'G']
Cost: 6

...Program finished with exit code 0
Press ENTER to exit console.
```